

Examen POO – 18 februarie 2025

1p din oficiu

1. (0,5p) Definiți conceptul de **încapsulare** în Programarea Orientată pe Obiecte. Dați un exemplu simplu în JAVA.
2. (0,5p) Scrieți în JAVA o clasă care să aibă o **variabilă instanță** care poate fi vizibilă numai în clasele care sunt în același pachet cu clasa respectivă.
3. (0,5p) Definiți în JAVA o clasă proprie **care nu poate fi derivată**.
4. (0,5p) Explicați ce se întâmplă în urma compilării și execuției următorului program JAVA:

```
public class A {  
    private static int nota = 4;  
  
    public void cresteNota() {  
        nota += 3;  
    }  
  
    public static void main(String[] args) {  
        cresteNota();  
        cresteNota();  
        System.out.println("La POO nota " + nota);  
    }  
}
```

5. (0,5p) Explicați ce se întâmplă în urma compilării și execuției următorului program JAVA:

```
interface A {  
    public default void mesaj() {  
        System.out.println("Sesiune de restante!");  
    }  
}  
  
public class B implements A {  
    public void mesaj() {  
        System.out.println("Examen la POO!");  
    }  
  
    public static void main(String[] args) {  
        A obj = new A();  
        obj.mesaj();  
    }  
}
```

6. (0,5p) Definiți în JAVA o clasă proprie, cu o metodă care aruncă o **excepție neverificată**.
7. (0,5p) Dați un exemplu simplu în JAVA în care **firul de lucru principal așteaptă** până când un alt fir de lucru (*thread*) își termină execuția.

8. (6x0,25p=1,5p) Scrieți pe 6 rânduri separate rezultatele afișate la ieșirea standard obținute în urma execuției următorului program JAVA:

```
class A extends B {
    static {
        y *= 5;
    }
    String x = "Examen ";
    public A(String x) {
        super(x);
        y *= 3;
    }
    public String method() {
        return x+"Restanta ";
    }
}
```

```
class B {
    String x = "POO ";
    static int y = 10;
    public B(String x) {
        y /= 2;
        x = "Sesiune ";
    }

    public String method() {
        return x+"Iarna ";
    }
}
```

```
public class C {
    public static void main (String[] args) {
        B b = new A("OK");
        System.out.println(A.y);
        A a = new A("OK");
        System.out.println(a.equals(b));
        System.out.println(b.x);
        System.out.println(b.method());
        System.out.println(b instanceof B);
        System.out.println(b.getClass().getName());
    }
}
```

9. (4p) Un magazin de IT vinde calculatoare, acestea fiind de două tipuri: laptop sau desktop. Pentru fiecare calculator se reține numărul unic al plăcii de bază, tipul de procesor, cantitatea de memorie RAM, capacitatea de stocare și prețul. Pentru un sistem desktop se reține în plus dacă este de tip server, iar pentru un laptop cât cântărește.

Respectând paradigma programării orientată pe obiecte, identificați entitățile, relațiile dintre acestea, desenați diagrama de clase UML detaliată (0,5p) și implementați în JAVA un program care să rezolve următoarele cerințe:

- (0,5p) Crearea unui magazin de IT care poate stoca calculatoarele pe care urmează să le vândă într-o colecție care este în permanență ordonată descrescător după prețul calculatoarelor
- (0,5p) Crearea de laptop-uri și desktop-uri prin citirea tuturor informațiilor de la intrarea standard
- (1p) Scoaterea la vânzare a unui nou calculator: acesta va fi stocat în cadrul colecției numai dacă numărul plăcii de bază nu se regăsește deja în colecție; în caz contrar calculatorul nou nu va fi stocat în colecție și se va arunca o excepție verificată cu mesajul "Calculator duplicat!"
- (0,5p) Determinarea numărului de laptop-uri scoase la vânzare care au procesor "Intel" și cântăresc cel mult 2 kg
- (0,5p) Afișarea în ordinea descrescătoare a prețului a tuturor detaliilor despre fiecare calculator scos la vânzare
- (0,5p) Scrieți o clasă cu o metodă *main* în care creați mai întâi un magazin de IT, iar apoi executați toate cerințele specificate mai sus astfel încât să utilizați fiecare funcționalitate cel puțin o dată